

2025 中山大学程序设计新手赛

Sun Yat-sen University

April 13, 2025

比赛开始前禁止翻阅试题

Problem A. 时间复杂度

输入文件:	standard input
输出文件:	standard output
时间限制:	1 second
空间限制:	256 megabytes

在比赛中你可能会遇见复杂度上的问题，例如一个算法程序运行时间过长，所以在实现算法前，我们会估算算法的时间复杂度。

关于复杂度，前人有很丰富的理论，这里我们只用最简洁的方式介绍时间复杂度。同一个算法在不同的计算机上运行的速度会有一定的差别，并且实际运行速度难以在理论上进行计算，实际去测量又比较麻烦，所以我们通常考虑的不是算法运行的实际用时，而是算法运行所需要进行的基本操作的数量。

影响时间复杂度的最主要因素是循环，我们通常可以通过估算指令循环进行的次数来得到一种时间复杂度。如以下以 n 和 m 为问题规模的 C 语言代码片段：

```
int k = 0;
for (int i = 0; i < n; ++ i)
  for (int j = 0; j < m; ++ j)
    ++ k;
```

根据乘法原理，其中的瓶颈指令 `++ k;` 被执行了 $n \times m$ 次，我们用 $O(nm)$ 表示其时间复杂度，其中大 O 表示指令执行次数的渐进上界。

通常来说，如果在题目条件下代码最多需要执行 10^8 量级的指令，则实际评测程序会运行约 $1s$。本次比赛的题目大多是以 $1s$ 为时间复杂度上限，如果你的算法需要执行 10^8 量级的指令，那么仍有机会通过，但是如果达到了 10^9 量级及以上，则可能要考虑重新设计算法或者改善常数部分了。

注意数据的输入输出会消耗比较大的时间，例如 `C` 风格的输入函数 `scanf` 在 $1s$ 内能读入 10^6 量级的整数，`printf` 类似。`C++` 的 `cin/cout` 与 `Python` 的 `input/print` 开销会更大，所以如果这些方法输入可能会超时，则请转用 `C` 风格的 `scanf` 输入。

Input

本题没有输入数据。

本题需要你阅读下列代码，并根据时间复杂度的增长速率进行排序，其中需要将增长速率较慢（也就是实际运行时较快）的代码排在前面。以下代码均以正整数 n 为问题规模。

```
// ID of this code is A
int k = 0;
for (int i = 1; i <= n; ++ i)
  for (int j = i; j <= n; ++ j)
    ++ k;
```

```
// ID of this code is B
int k = 0;
for (int i = 1; i <= n; ++ i)
  for (int j = 1; i * j <= n; ++ j)
    ++ k;
```

```
// ID of this code is C
int k = 0;
```

```
for (int i = 1; i <= n; ++ i)
  for (int j = 1; j * j <= n; ++ j)
    ++ k;
```

```
// ID of this code is D
int fib(int n) {
  if (n <= 1) {
    return 1;
  }
  return (fib(n - 1) + fib(n - 2)) % 998244353;
}

int result = fib(n);
```

```
// ID of this code is E
int arr[n + 1];

int fib(int n) {
  if (n <= 1) {
    return 1;
  }
  if (arr[n] != -1) {
    return arr[n];
  }
  return arr[n] = (fib(n - 1) + fib(n - 2)) % 998244353;
}

for (int i = 1; i <= n; ++ i)
  arr[i] = -1;
int result = fib(n);
```

Output

输出一行一个字符串，其应当包含大写字母 `A`，`B`，`C`，`D` 和 `E` 恰好一次。例如，输出 `ABCDE` 代表认为在 n 足够大时代码执行所需的时间从短到长依次为：程序 `A`，程序 `B`，程序 `C`，程序 `D` 和程序 `E`。

Problem B. 完美洗牌

输入文件:	standard input
输出文件:	standard output
时间限制:	1 second
空间限制:	256 megabytes

小 F 很擅长洗牌，特别是完美洗牌。小 F 在每次完美洗牌时，会将牌堆对半分成两份，然后将它们再插回一堆。具体来说，他会将一个包含 n（n 是偶数）张牌的牌堆，从堆顶到堆底的每个位置按 $1 \sim n$ 标号，每次洗牌后：

- 原位于 $1 \le i \le \frac{n}{2}$ 位置的牌将会移动到第 $2i-1$ 位；
- 原位于 $\frac{n}{2} < i \le n$ 位置的牌将会移动到第 $2i-n$ 位。

以包含六张牌的牌堆为例，若其初始顺序为 `123456`（堆顶至堆底），经过一次完美洗牌后，新的顺序将变为 `142536`。

现在，小 F 有一堆没有花色的扑克牌（可能有超过四张同一牌面的牌，比如五张 `2`），按照堆顶到堆底给定每张牌的牌面，他想让你帮他求出最少要洗牌多少次才能回到初始的状态。

Input

本题采用多组测试数据，第一行包含一个整数 T $(1 \le T \le 10)$，代表数据组数。

在每组数据中：

第一行包含一个整数 n $(1 \le n \le 1000)$，代表扑克牌的张数。保证 n 是偶数。

第二行包含一个长度为 n 的字符串，保证每个字符是 `23456789JQKA` 中的一个，代表从堆顶到堆底每张牌的牌面。

Output

输出 T 行，每行一个整数，代表最少需要的洗牌次数。

Example

standard input	standard output
4 4 JQKA 8 23456789 12 223222333232 8 AQAQAQAQ	2 3 2 3

Note

对于第三组测试数据，两次洗牌的结果分别为：

$$223222333232 \xrightarrow{\texttt{shuffle}} 232333222322 \xrightarrow{\texttt{shuffle}} 223222333232$$

对于第四组测试数据，三次洗牌的结果分别为：

$$\texttt{AQAQAQAQ} \xrightarrow{\texttt{shuffle}} \texttt{AAQQAAQQ} \xrightarrow{\texttt{shuffle}} \texttt{AAAAQQQQ} \xrightarrow{\texttt{shuffle}} \texttt{AQAQAQAQ}$$

Problem C. 游览路线

输入文件:	standard input
输出文件:	standard output
时间限制:	1 second
空间限制:	256 megabytes

一个游览景点可以抽象成一个包含 n 个点和 m 条边的带权无向连通图，第 i 条边连接点 u_i 和 v_i，其权值为 w_i。游客想从起点 s 点进入，并从终点 t 点离开，而这一路上的高兴值为边权的按位与。形式化来说，假设这位游客自起点 s 经过编号分别为 $i_1, i_2, \ldots, i_k$ 的边并到达终点 t，那么这条路径对应的高兴值为 w_{i_1} & w_{i_2} & $\ldots$ & w_{i_k}（这里的 & 代表按位与）。

现在给出这张无向图，以及起点 s 和终点 t，请求出在最佳的方案下，最大的高兴值可能是多少。

Input

第一行包含两个整数 n, m $(1 \le n \le 10^5, 1 \le m \le 5 \times 10^5)$，代表图的点数和边数。

第二行包含两个整数 s, t $(1 \le s, t \le n, s \ne t)$，分别代表游览的起点和终点。

接下来 m 行，第 i 行包含三个整数 u_i, v_i, w_i $(1 \le u_i, v_i \le n, 1 \le w_i \le 10^9, u_i \ne v_i)$，代表一条连接点 u_i 和 v_i 且权值为 w_i 的边。**图中可能出现重边，也就是若干条边连接了同一对节点的情况。**数据保证任意两个点都可以通过这些路径互相到达。

Output

一行一个整数，代表最大的高兴值。

Example

standard input	standard output
5 10 5 1 4 1 8 4 5 3 4 3 4 4 2 10 1 4 4 1 2 4 3 5 8 1 5 4 5 2 7 1 2 5	5

Note

对于样例，可以选择从 5 号点通过第 9 条边走到 2 号点，随后通过第 10 条边走到 1 号点。整条路径对应的高兴值为 $7 \oplus 5 = 5$，可以证明这是最大的高兴值。

本题输入数据量较大，请使用较快的输入方式。

Problem D. 靶场

输入文件:	standard input
输出文件:	standard output
时间限制:	1 second
空间限制:	256 megabytes

在因为糟糕的枪法被朋友嘲笑后，小 F 决定去靶场苦练自己的枪法。靶场可以看作一个圆，其圆周被均匀分为 c 个瞄准点，按照顺序标号为 $0, 1, \ldots, c-1$，小 F 站在中心点且不会移动，初始时（对应第 0 秒）面朝 0 号瞄准点。小 F 每次练习会打 n 个靶，其中第 i 个靶会在第 i 秒出现在第 a_i 个瞄准点。小 F 预先知道了所有靶子的位置，因此规定：只有**在靶子出现的一瞬间开枪并命中**，才会认为自己中靶。

若小 F 每秒钟只能转动最多 s 个瞄准点的角度，那么在最优情况下，他能命中多少个靶（假设他只要面朝着某个靶所在的方向，就一定能命中这个靶）？

Input

本题采用多组测试数据，第一行包含一个整数 T $(1 \le T \le 10)$，代表输入数据组数。

在每组数据中：

第一行包含三个整数 n, c, s $(1 \le n \le 1000, 1 \le s < c \le 10^9)$，分别表示靶子个数，瞄准点个数和最大转动速度。

接下来 n 行，第 i 行包含一个整数 a_i $(0 < a_i < c)$，代表第 i 个靶子所在的瞄准点。

Output

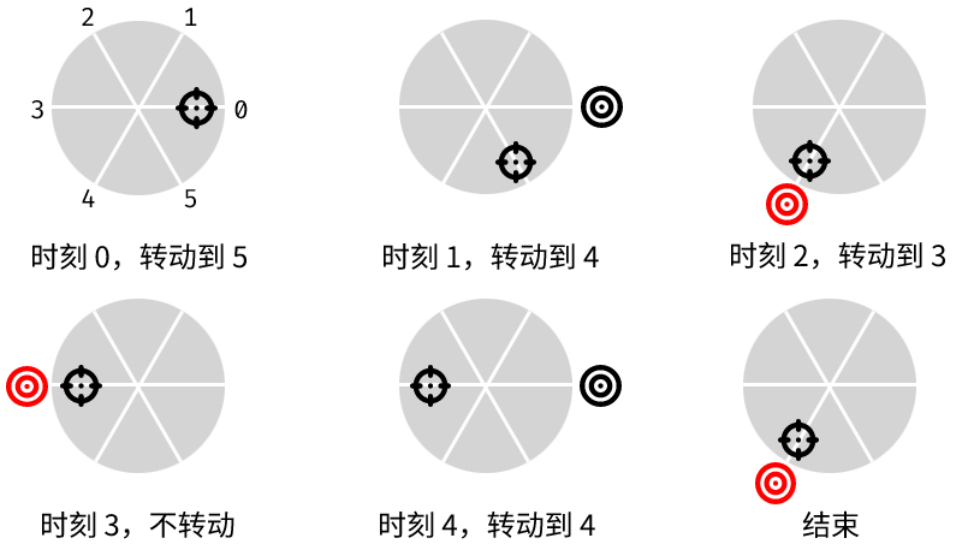
对每组数据，输出一行一个整数，代表在最优情况下能命中的靶子数量。

Example

standard input	standard output
2 5 6 1 0 4 3 0 4 11 45 14 13 40 30 1 32 5 22 39 4 19 35	3 6

Note

对于第一个测试数据，以下是一种可能的转动方案：



Problem E. Soft-取模

输入文件:	standard input
输出文件:	standard output
时间限制:	1 second
空间限制:	256 megabytes

小 W 在课堂上学习了如何进行取模。他感觉取模操作实在太单调了，于是创造了一种新型的取模方案，名为 Soft-取模。

具体而言，Soft-取模需要一个正整数 x，一个正整数区间 $[l, r]$，以及一个步长 d，且需要满足 d 是 $r - l$ 的因数。Soft-取模会进行共 $\frac{r-l}{d} + 1$ 次取模操作：在第 i 次操作时，将 x 赋值为 $x \bmod (r - d * (i - 1))$。这相当于让 x 依次对 r，$r - d$，$\ldots$，$l + d$ 和 l 取模。

小 W 希望知道，在每次取模后 x 的新值的总和是多少。小 W 会进行若干次 Soft-取模操作，并希望你可以对每次 Soft-取模操作求出对应的总和。

Input

第一行包含一个整数 T $(1 \le T \le 10^5)$，表示 Soft-取模的次数。

接下来 T 行，每行包含四个整数 x, l, r, d $(1 \le x \le 10^9, 1 \le l \le r \le 10^9, 1 \le d \le 10^9, d \mid (r - l))$，表示一次 Soft-取模操作。

Output

输出 t 行，每行包含一个整数，表示对应的 Soft-取模在每次取模后的新值总和。

Example

standard input	standard output
3 7 1 5 2 1919810 114 514 10 749999999 1 500000000 1	4 820 62499999999999999

Note

对于样例的第一次 Soft-取模，有 $x = 7$，$l = 1$，$r = 5$ 和 $d = 2$，共进行三次取模。

1. 第一次取模时将 $x = 7$ 对 5 取模，新值为 2；
2. 第二次取模时将 $x = 2$ 对 3 取模，新值为 2；
3. 第三次取模时将 $x = 2$ 对 1 取模，新值为 0。

故 x 的新值的总和为 $2 + 2 + 0 = 4$。

Problem F. 商店

输入文件:	standard input
输出文件:	standard output
时间限制:	1 second
空间限制:	256 megabytes

小 F 最近沉迷某款四字游戏，每天早上八点醒来第一件事就是打开每日商店看看是否刷出心仪的皮肤。他的好兄弟 ZZ 看他刷不出金整天愁眉不展，打算帮助他写个程序预测一下。

假设一共有 n 种类型的枪械，第 i 类枪械一共有 a_i 种皮肤，而小 F 心仪的只有其中的 b_i 种。商店每天只刷新一种皮肤，且如果小 F 购买过某类枪械的皮肤，那么这类枪械的所有皮肤将不会再次在商店里刷出。商店里刷出的皮肤会在所有可刷新的皮肤中等概率随机，只要刷新出小 F 心仪的皮肤，他就会购买。具体的，假设现在小 F 没购买过的枪械的集合为 S ，那么每日商店刷新的物品将在 $\sum_{i\in S} a_i$ 个皮肤中等概率随机。

现在 ZZ 想让你帮忙写个程序预测小 F 期望需要多少天才能为每类枪械购买一件皮肤。

Input

第一行包含一个整数 n $(1 \le n \le 19)$，代表枪械类型总数。

接下来 n 行，第 i 行包含两个整数 a_i, b_i $(1 \le b_i \le a_i \le 100)$，分别代表第 i 类枪械的皮肤数量和其中小 F 心仪的皮肤数量。

Output

输出一行一个整数，若期望天数的最简分数为 $\frac{P}{Q}$，那么你应当输出 $P \cdot Q^{-1} \bmod (10^9+7)$。

Examples

standard input	standard output
3 3 1 3 1 3 1	9
19 65 27 55 3 32 14 59 1 75 71 96 84 69 49 68 59 98 73 85 73 20 3 14 13 75 62 86 46 35 10 40 20 13 3 25 12 85 48	2374748

Problem G. 过河卒

输入文件:	standard input
输出文件:	standard output
时间限制:	1 second
空间限制:	256 megabytes

棋盘上有黑红两方的过河卒，需要走到底线。过河卒行走的规则是可以向左移动一格，向右移动一格或者向前移动一格。然而，这一题没有对方的棋子拦截你的过河卒，双方需要做的反而是配合让所有过河卒走到底线。

让我们将问题进行简化。给定一个 n 行 m 列的狭长棋盘，其中 m 的取值只会是 2 或 3。棋盘的第 1 行有 m 个红子，第 n 行有 m 个黑子。在一步棋中，你可以选择棋盘上的任意一个子，并将其移动到相邻且没有子的位置上。两个位置相邻，当且仅当二者在同一行且列数差恰好为 1，或者二者在同一列且行数差恰好为 1。

现在要将所有黑子移动到第 1 行，所有红子移动到第 n 行，请你算出完成目标所需的最少步数。

Input

第一行包含两个整数 n, m $(6 \leq n \leq 10^9, 2 \leq m \leq 3)$，代表棋盘的大小。

Output

输出一行一个整数，代表答案。

Examples

standard input	standard output
6 2	22
6 3	34

Problem H. 三仙归洞

输入文件:	standard input
输出文件:	standard output
时间限制:	3 seconds
空间限制:	512 megabytes

现在有 n 位杂技师同时进行表演，每个人的面前有一张桌子，桌子上有一根筷子、两个倒扣的碗和若干小球。其中第 i 个人左手边的碗内有 a_i 个小球，右手边的碗内有 b_i 个小球，在碗外面的桌面上方，还有 c_i 个小球。除此之外，现场没有任何其他的小球，即小球的位置和数量情况可以用三个序列 $\{a_n\},\{b_n\},\{c_n\}$ 来表示。

现在有若干次表演。一次表演可以被形式化为，选定一个区间 $[l,r]$，然后选定 $\{a_n\},\{b_n\},\{c_n\}$ 中的两个序列，不妨记其为 $\{p_n\},\{q_n\}$，接着对所有 $j\in[l,r]$，交换 p_j 和 q_j 的值。

作为这场表演的观众，你因为拥有透视功能，且观察相当细致，所以知道了以上的所有信息。出于好奇，你会进行若干次观察。一次观察可以被形式化为，计算当前第 l 位杂技师到第 r 位杂技师左手边的碗内的小球总数，即 $\sum_{i=l}^{r} a_i$。

由于杂技师的数量众多，于是你决定用计算机编写一个程序来帮助你解答这个问题。为了方便表述，我们记表演和观察总共有 m 次。

Input

第一行包含两个整数 n,m $(1\le n\le 5\times 10^5, 1\le m\le 8\times 10^5)$，分别表示杂技师的数量，以及表演与观察的总次数。

接下来三行，每行包含 n 个整数，分别表示三个序列 $\{a_n\},\{b_n\},\{c_n\}$ $(1\le a_i,b_i,c_i\le 2\times 10^3)$。

接下来 m 行，每行包含三个整数 opt,l,r $(0\le opt\le 3, 1\le l\le r\le n)$，表示一次表演或观察。具体而言：

- 当 $opt=0$ 时，表示当前进行了一次表演，选择的 p,q 序列分别为 a 序列和 b 序列；
- 当 $opt=1$ 时，表示当前进行了一次表演，选择的 p,q 序列分别为 a 序列和 c 序列；
- 当 $opt=2$ 时，表示当前进行了一次表演，选择的 p,q 序列分别为 b 序列和 c 序列；
- 当 $opt=3$ 时，表示当前进行了一次观察。

Output

对于每次观察，输出一行一个整数，表示这次观察的计算结果。

Example

standard input	standard output
8 10 486 249 12 175 432 1074 1054 677 1325 159 941 1090 486 1324 989 1208 331 801 1232 930 1004 179 1073 99 3 5 7 2 5 6 1 4 5 2 7 8 3 5 8 1 5 7 0 2 8 2 2 5 0 3 5 3 1 8	2560 3291 3889

Note

本题输入数据量较大，请使用较快的输入方式。

Problem I. 挑选座位

输入文件:	standard input
输出文件:	standard output
时间限制:	2 seconds
空间限制:	256 megabytes

中午下课的时候，松韬园里总是有很多人，小 Z 一般会挑选一个周围的人尽量少的座位入座。但松韬园很大，小 Z 有时候不能在短时间里找到合适的座位，所以他决定统计每个座位周围的人数。

松韬园里一共有 n 个座位，第 i 个座位上已经坐了 a_i 个人。这 n 个座位被 $n-1$ 条边相连，且保证这些座位可以互相到达。两个座位是相邻的，当且仅当这两个座位被一条边相连。一个座位周围的人数被定义为**这个座位的人数和与这个座位相邻的所有座位的人数**之和。

小 Z 会在 q 个单位时间里统计松韬园座位周围的人数，在每个单位时间里，可能发生两种事件：

- 第 x 个座位的人数会变成 y，即令 $a_x \leftarrow y$；
- 小 Z 想要知道第 x 个座位周围的人数。

小 Z 希望你能帮他解决这个问题。

Input

第一行包含两个整数 n, q $(1 \le n, q \le 5 \times 10^5)$，分别代表座位数量和单位时间数量。

第二行包含 n 个整数 a_i $(0 \le a_i \le 10^9)$，代表初始情况下每个座位上的人数。

接下来 $n-1$ 行，每行包含两个整数 a, b $(1 \le a, b \le n, a \ne b)$，表示第 a 个座位和第 b 个座位被一条边相连。

接下来 q 行，每行的最开始为一个整数 opt $(1 \le opt \le 2)$，表示事件的类型。随后：

- 若 $opt = 1$，则接下来包含两个整数 x, y $(1 \le x \le n, 0 \le y \le 10^9)$，表示第 x 个座位的人数会变成 y；
- 若 $opt = 2$，则接下来包含一个整数 x $(1 \le x \le n)$，表示小 Z 想要知道第 x 个座位周围的人数。

Output

对于每个 $opt = 2$ 的事件输出一行一个整数，表示询问的答案。

Example

standard input	standard output
5 3 1 2 3 4 5 1 2 2 3 2 4 4 5 2 2 1 2 3 2 2	10 11

Note

本题输入数据量较大，请使用较快的输入方式。

Problem J. 高速公路

输入文件:	standard input
输出文件:	standard output
时间限制:	3 seconds
空间限制:	512 megabytes

G 省的高速公路系统可以看成包含 n 个点和 m 条边的带权无向图，每条边代表一段高速公路，而每个节点代表一个城市。每段公路都可以双向通行，第 i 段公路连接 u_i, v_i 这两个城市，且长度为 w_i 公里。

现在小 Z 正在 G 省通勤，小 Z 的车每行驶一公里消耗一单位的油。任意一个城市中存在加油站可以把车的油加满，忽略小 Z 在城市中行驶的油耗。现在小 Z 计划了 q 次出行，第 i 次出行从城市 x_i 开始，如果他想要能到达至少 c_i 个不同的城市（包括 x_i），请问他的车的油箱的容量至少为多少?

Input

输入的第一行包含两个整数 n, m $(1 \le n \le 10^5, 1 \le m \le 2 \times 10^5)$，分别代表城市的数量和高速公路的数量。

接下来 m 行，第 i 行包含三个整数 u_i, v_i, w_i $(1 \le u_i, v_i \le n, u_i \ne v_i, 1 \le w_i \le 2 \times 10^6)$，代表第 i 条高速公路连接 u_i, v_i 两个城市，长度为 w_i 公里。**可能会有多条高速公路连接一对相同的城市。**

接下来一行包含一个整数 q $(1 \le q \le 10^6)$，代表出行次数。

接下来 q 行，第 i 行包含两个整数 x_i, c_i $(1 \le x_i \le n, 2 \le c_i \le n)$，表示一次询问。

Output

输出 q 行，每行包含一个整数，代表最小油箱容量。

Example

standard input	standard output
5 7 3 5 13 1 2 8 1 4 6 5 1 9 1 2 8 3 4 2 5 1 15 5 3 2 5 2 4 3 2 4 2 5	2 9 6 8 9

Note

本题输入数据量较大，请使用较快的输入方式。

Problem K. 分配系统

输入文件:	standard input
输出文件:	standard output
时间限制:	1 second
空间限制:	256 megabytes

小 T 在学习操作系统原理后心血来潮，想要自己实现一个分配系统。这个分配系统由若干个栈组成，其中每个栈的初始容量为 S 字节。

在一个任务中，分配系统需要连续处理 n 次分配请求。对于第 i 次请求，其需要得到连续的 a_i 字节的空间以进行操作，此时小 T 会选择**剩余容量最大**的栈，并尝试在这个栈的顶部分出连续的 a_i 字节空间。如果这个分配系统在中途出现空间不够的情况，则认为任务失败，否则任务成功。

小 T 想要知道，在所有的栈初始均为空的情况下，为了完成任务，至少需要使用多少个栈?

Input

输入的第一行包含两个整数 n, S $(1 \le n \le 10^5, 1 \le S \le 10^9)$，分别表示请求次数和栈的初始大小。

第二行包含 n 个整数 a_i $(1 \le a_i \le S)$，表示每次请求所需的空间大小。

Output

输出一行一个整数，表示最少所需的栈数量。

Example

standard input	standard output
8 20 4 6 5 1 10 8 6 9	4

Note

对样例数据，以下是在使用三个栈时对应的分配信息:

i	1	2	3	4	5	6	7	8
a_i	4	6	5	1	10	8	6	9
第一个栈的大小	<u>20</u>	16	16	<u>16</u>	<u>15</u>	5	5	5
第二个栈的大小	20	<u>20</u>	14	14	14	14	<u>14</u>	8
第三个栈的大小	20	20	<u>20</u>	15	15	<u>15</u>	7	7

可以发现在最后一次分配时，所有栈均没有足够的容量，故使用三个栈时无法完成任务。

Problem L. 提前修改

输入文件:	standard input
输出文件:	standard output
时间限制:	1 second
空间限制:	256 megabytes

Alice 和 Bob 在进行游戏！具体来说，现在桌面上有 n 堆石子，其中第 i 堆石子有 a_i 个，先后手轮流操作，每次操作者需要选择一堆非空的石子，并拿走至少一颗石子，取完最后一颗石子的玩家取得胜利。由于 Alice 和 Bob 都是绝顶聪明的，所以他们总会采用最优策略进行游戏。

喜欢研究博弈论的你当然知道，这是典型的 Nim 游戏，当双方均采用最优策略时，先手在 $a_1 \oplus a_2 \oplus a_3 \oplus \cdots \oplus a_n = 0$ 时必败，否则必胜，其中 $\oplus$ 表示异或符号，也即 C/C++ 语言中的 `xor` 运算符。只通过一点计算，你就预测出了最后的结果，于是你决定在游戏进行前，尝试对当前局面做操作。

一次操作定义为，选择两个任意的正整数 $l, r (1 \le l \le r \le n)$，然后对**所有**在 $[l, r]$ 中的正整数 i，将 a_i 异或上 i，即 $a_i \leftarrow a_i \oplus i$。修改将通过宇宙射线完成，所以你无需考虑修改耗时太久而影响 Alice 和 Bob 的游戏进行。

你想要知道，是否存在有限次操作的方案，使得局面能够转变为先手必败态。由于你追求完美，所以更进一步地，如果存在这样的操作方案，你还希望所需的操作次数能够**最小化**。你决定写一个程序来解答这个问题。

Input

第一行包含一个整数 n $(1 \le n \le 10^6)$，表示石子堆数。

第二行包含 n 个整数 a_i $(1 \le a_i \le 10^9)$，表示每堆石子的初始个数。

Output

若不存在有限次的操作方案，使得局面能够转为先手必败态，则输出 `-1`。

否则先输出一行一个整数 m，表示所需的最小操作次数。

接下来输出 m 行，第 i 行包含两个整数 l_i 和 r_i，表示第 i 次操作的区间是 $[l_i, r_i]$。你的输出需要满足 $1 \le l_i \le r_i \le n$。

如果存在多种操作次数最小的合法方案，输出任意一种即可。

Examples

standard input	standard output
5 20 24 8 7 2	1 4 5
8 21 52 11 25 41 67 77 10	-1

Note

对于第一个样例，我们可以选择仅修改一次，并选择区间 $[4, 5]$ 进行操作。操作后 $a_4 = 3, a_5 = 7$，满足 $a_1 \oplus a_2 \oplus a_3 \oplus a_4 \oplus a_5 = 20 \oplus 24 \oplus 8 \oplus 3 \oplus 7 = 0$，此时局面变为先手必败态。**需要注意的是**，这只是其中一种最优方案，即使你的答案和样例输出不同，只要你的答案是任意一种最优方案，你的答案都会被判定为正确。

Problem M. 尖塔攻击

输入文件:	standard input
输出文件:	standard output
时间限制:	1 second
空间限制:	256 megabytes

在一个平面坐标系上有 k 座尖塔，第 i 座尖塔被放置在整点 (x_i, y_i) 上，并且满足 $1 \le x_i \le n$ 且 $1 \le y_i \le m$。尖塔两两之间会产生电流，对它们的中点进行一次攻击。需要注意的是，虽然尖塔被放置在整点上，但其中点并不一定落在整点上，此时的攻击依然是生效的。

如果一个位置被攻击了**至少两次**，那么这个位置就会被损坏。你需要检查是否存在受损的位置，如果有，还需要给出其中任意一个位置的坐标。

Input

本题采用多组测试数据，第一行包含一个整数 T $(1 \le T \le 10)$，表示数据组数。

在每组数据中：

第一行包含三个整数 n, m, k $(1 \le n, m \le 1000, 1 \le k \le \min(n \times m, 10^5))$。

接下来的 k 行，第 i 行包含两个整数 x_i, y_i $(1 \le x_i \le n, 1 \le y_i \le m)$，表示第 i 座尖塔的位置坐标。**两座尖塔的位置可能相同，此时它们的中点视为尖塔所在的位置。**

Output

对于每组数据，输出一行：

- 如果存在至少一个损坏的位置，则输出 `YES a b`，其中 (a, b) 代表任意一个受损的位置坐标；
- 否则，输出 `NO`。

Example

standard input	standard output
2 3 3 4 1 1 1 2 2 2 2 3 5 5 7 1 5 2 3 1 1 1 3 1 2 5 1 3 3	YES 1.5 2.0 NO

Note

对于第一组数据：第一座尖塔和第四座尖塔形成的电流会在 $(1.5, 2)$ 处造成攻击，而第二座尖塔和第三座尖塔形成的电流也会在 $(1.5, 2)$ 处造成攻击，故我们认为 $(1.5, 2)$ 位置受损。